

PROMPT

AI

Prompt Engineering Interview Questions

33 Q&As to ace your next AI interview



Chandra Sekhar
AI Engineer & Trainer

What is prompt engineering?

Prompt engineering is the practice of designing clear and structured instructions so that a language model produces the desired output. Instead of changing the model, you change how you ask the question. Example: 'Explain APIs' vs 'Explain APIs to a beginner in simple words with one real-world example'—the second produces a much better and more targeted answer.

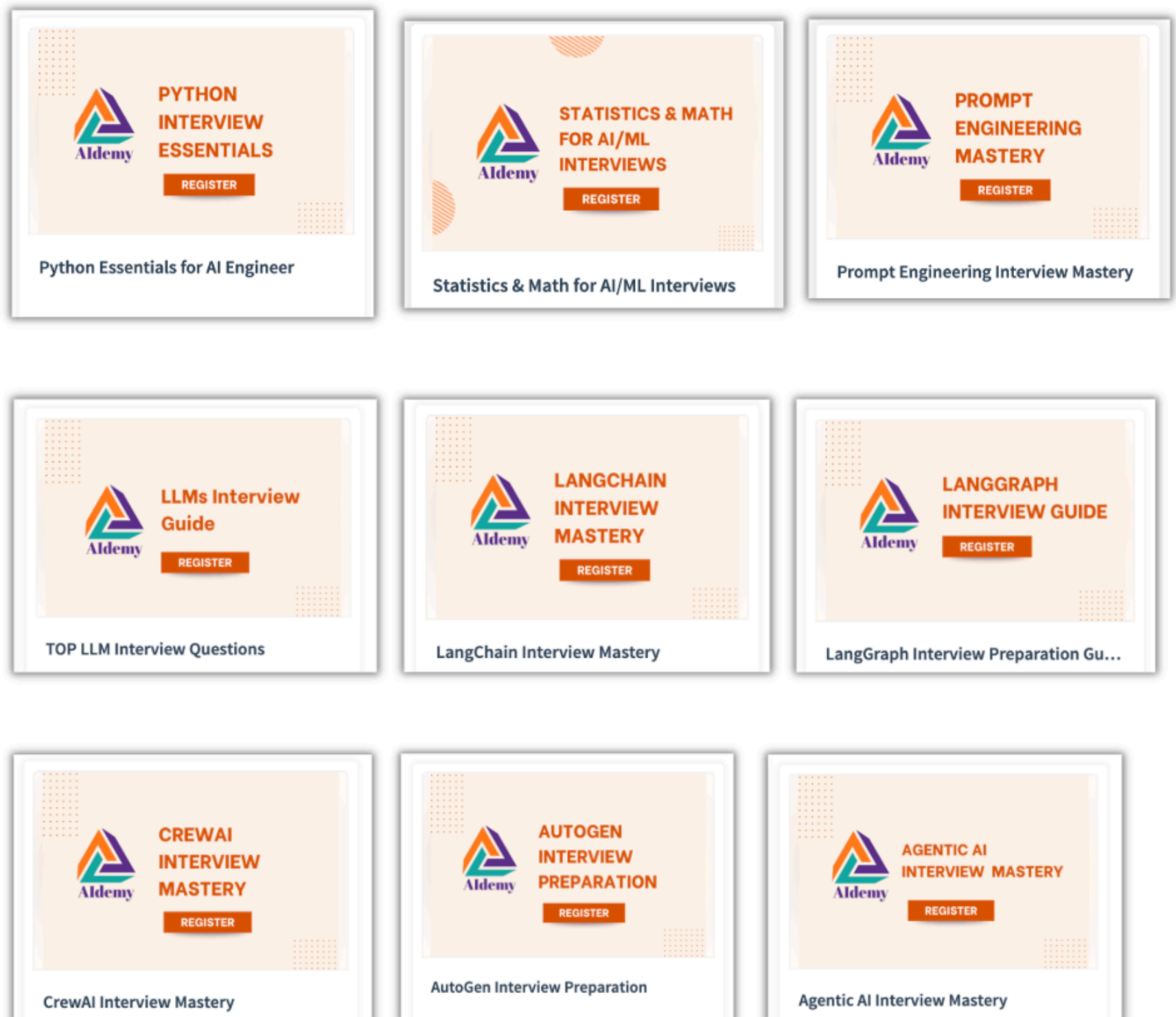
What is a zero-shot prompt? Give an example.

A zero-shot prompt asks the model to perform a task without providing any examples. The model relies entirely on its general understanding. Example: 'Classify the sentiment of the following text as Positive or Negative: I really enjoyed the movie.' No examples are given, yet the model can still respond correctly. Zero-shot prompts are useful when the task is simple, you want fast responses, or examples are not available.



I have created Ultimate AI Engineer Interview Bootcamp

13 Courses | Structured Learning | Interview Mastery



ENROLL NOW

*Link in caption

What is few-shot prompting and why do we use it?

Few-shot prompting provides a small number of examples before asking the model to perform a task. This helps the model understand the expected format, the type of reasoning required, and the style of the answer. It is used when zero-shot results are inconsistent, the task has specific formatting rules, or higher accuracy is needed.

What is the difference between a system prompt and a role prompt?

System prompt defines global behavior and rules for the model (e.g. 'You are a helpful assistant that answers concisely and avoids speculation'). Role prompt assigns a specific persona or responsibility (e.g. 'You are a senior Python instructor teaching beginners'). The system prompt controls how the model behaves overall; the role prompt controls who the model acts as.



What are some best practices for writing good prompts?

Be clear and specific; state the task explicitly; define output format if needed; add constraints (length, tone, source); avoid vague or open-ended instructions; tell the model what to do if information is missing. Example: 'Explain recursion in simple terms. Use one real-world analogy. Limit the answer to 5 sentences.' Clear constraints lead to predictable and useful outputs.

How do you measure whether a prompt is effective?

A prompt is effective if it consistently produces correct answers, relevant responses, desired format and tone, and minimal hallucinations. Measure by testing with multiple inputs, checking consistency across runs, comparing outputs before and after prompt changes, and collecting user feedback. If changing a prompt reduces incorrect or irrelevant answers, the new prompt is more effective.



What does the temperature setting do in an LLM?

Temperature controls how random or creative the model's responses are. Low (0–0.3) → more predictable, factual answers. Medium (0.5–0.8) → balanced creativity and accuracy. High (1.0+) → more diverse but riskier outputs.

Temperature 0.2 tends to give the same answer every time; 0.9 gives different wording or ideas each time.

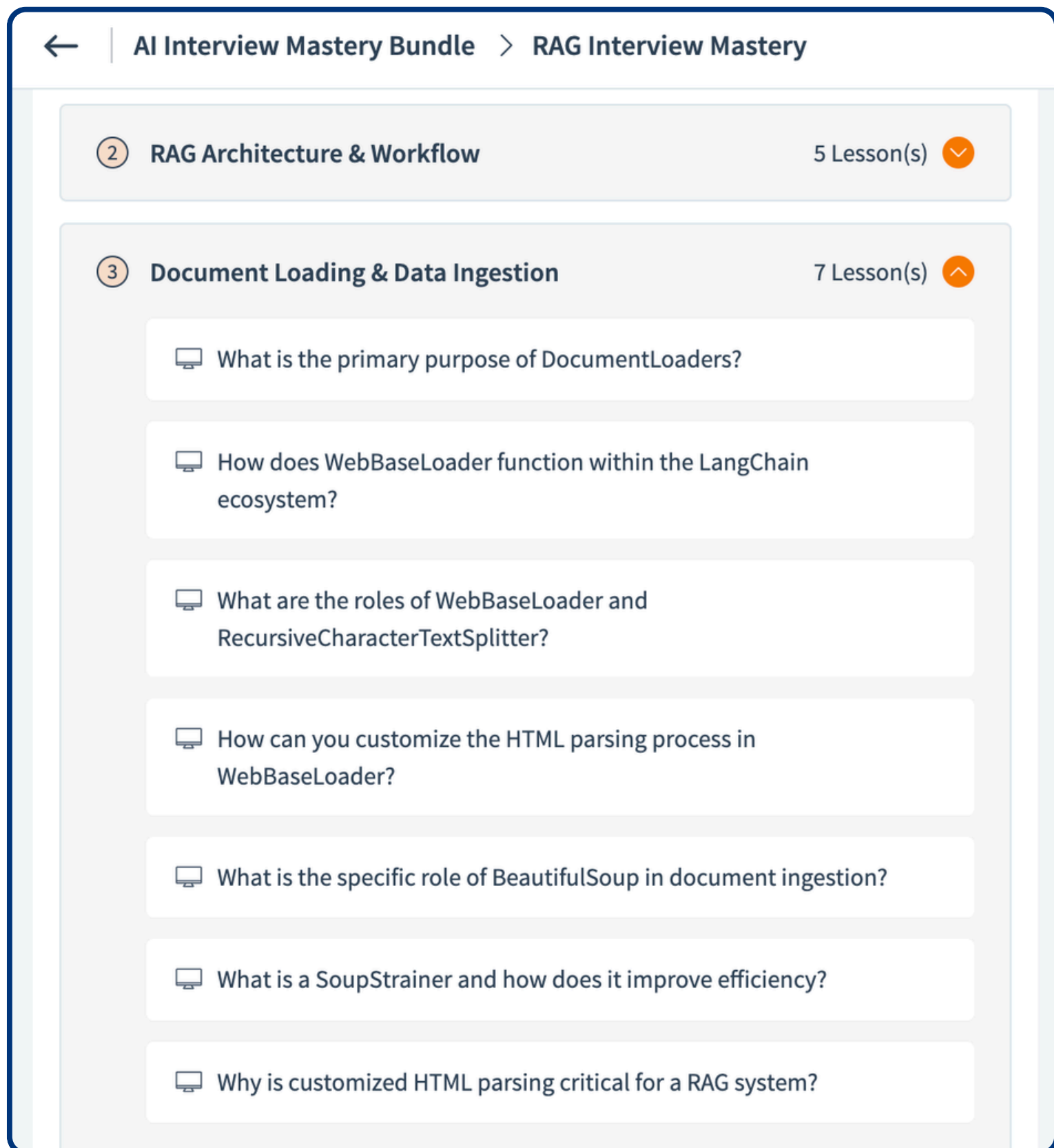
What is top-K and top-P sampling?

Both control which tokens the model is allowed to choose from. Top-K: the model selects from the top K most probable tokens (e.g. K=5 → only the 5 most likely next words). Top-P (nucleus): the model selects from tokens whose combined probability exceeds P (e.g. P=0.9 → tokens chosen until total probability reaches 90%). Top-K uses a fixed limit; Top-P adjusts dynamically.



AI Engineer Interview Bundle

Each course divided into section wise questions and answers



ENROLL NOW

*Link in caption

What should you do if the model repeats words or generates loops?

Lower the temperature; reduce top-K or top-P; add explicit stopping instructions; set a maximum output length; rephrase the prompt to be more specific. Example instruction: 'Do not repeat phrases. Provide the answer once in concise form.'

How can you fix hallucinations or inaccurate answers using prompts?

Constrain the model clearly: 'Answer only using the provided information'; 'If the answer is unknown, say I don't know'; avoid open-ended or speculative language; ask for sources or justification from context. Example: 'Use only the context below. If the answer is not present, respond with Not found in context.'



How do you handle long or multi-step tasks using prompts?

Break them into steps: explicitly ask for step-by-step reasoning; split the task into smaller sub-tasks; ask for intermediate outputs; summarize after each step if needed. Example: 'Step 1: Analyze the problem. Step 2: List possible solutions. Step 3: Choose the best solution and explain why.' This improves clarity, accuracy, and consistency.

What is Chain of Thought (CoT) prompting?

CoT asks the model to reason step by step instead of giving a direct answer. It helps the model show intermediate reasoning, which improves accuracy for logic, math, and multi-step problems. Example: 'Question: If a pen costs \$2 and I buy 3 pens, how much do I pay? Think step by step.' The model explains the steps before giving the final answer.



What is self-consistency prompting?

Self-consistency generates multiple reasoning paths for the same question and then selects the most common or consistent answer. Instead of trusting one chain of thought, the model samples several and compares results (e.g. generate 5 different step-by-step solutions, pick the answer that appears most often). This reduces random errors and improves reliability.

What is Tree of Thoughts (ToT)?

Tree of Thoughts extends Chain of Thought by exploring multiple reasoning paths in parallel, like branches of a tree. Instead of one linear chain, the model generates multiple possible next steps, evaluates them, and continues only with the most promising ones. For a planning problem, the model may explore Plan A, B, C and discard weaker plans early.



What is step-back prompting?

Step-back prompting asks the model to first think at a higher level before solving the problem in detail. It helps avoid getting stuck in details too early. Example: 'Step 1: What is the general principle behind this problem? Step 2: Now solve the specific question.' This improves reasoning for complex or unfamiliar problems.

What is the difference between Chain of Thought and ReAct prompting?

CoT focuses only on step-by-step reasoning; reasoning happens internally in one flow. ReAct (Reason + Act) combines reasoning with actions: the model thinks, then uses tools (search, calculator, retriever), then thinks again. ReAct flow: Think → Act (e.g. call search) → Observe results → Continue reasoning.



How does Tree of Thoughts improve on Chain of Thought?

CoT follows one reasoning path, which may be wrong. ToT explores multiple paths, evaluates them, and can backtrack if needed. If one path leads to a dead end, ToT can switch to another branch. This makes ToT more powerful for complex planning, optimization problems, and decision-making tasks.

What is ReAct prompting?

ReAct (Reason + Act) combines reasoning and action in a single loop. The model thinks about what to do next, takes an action (e.g. calling a tool), observes the result, and continues reasoning. Example: Thought: I need the latest policy details. Action: Search HR policy documents. Observation: Retrieved updated leave policy. Thought: I can now answer the question. Useful when answers require external data or tools.



What is Automatic Prompt Engineering (APE)?

APE is a technique where prompts are generated and optimized automatically instead of being written manually. The system creates multiple prompt candidates, tests them on sample inputs, compares the outputs, and selects the best-performing prompt. This reduces trial-and-error in prompt writing.

What is APE used for, and how do you judge the best prompt?

APE is used to improve accuracy, find more reliable prompt formats, and optimize prompts at scale. The best prompt is judged by correctness of outputs, consistency across examples, adherence to format, and lower hallucination rate. If Prompt A gives correct answers 90% of the time and Prompt B gives 70%, Prompt A is preferred.



How can variables be used in prompts to handle dynamic user input?

Variables allow prompts to change dynamically based on user input or system state. Placeholders are used inside prompts and filled at runtime. Example template: 'Answer the question using the context below. Context: {context} Question: {user_question}' This allows the same prompt structure to work for many different users and queries.

How can prompt engineering help create a customer support chatbot?

Define tone (polite, professional), behavior (no speculation, factual), escalation rules, and response format. Example: 'You are a customer support assistant. Answer politely and clearly. If the issue is not covered in the knowledge base, ask the user to contact support.' Well-designed prompts ensure consistent responses, fewer hallucinations, and better customer experience.



How can you design a prompt to extract structured data from messy text?

Clearly define what fields to extract and the expected output format: specify exact fields, define output structure, instruct the model to ignore irrelevant text. Example: 'Extract: Name, Email, Phone number. Return in JSON. Text: Hi, this is Rahul. rahul@gmail.com, 9876543210.' Clear instructions turn unstructured text into reliable structured data.

How can prompts be used to generate code in different programming languages?

Specify the programming language, the task, and constraints or style. The model then generates code accordingly. Example: 'Write a function in Python to check if a number is prime.' Changing to 'Write the same function in JavaScript' keeps the logic and adapts syntax. The logic stays the same; syntax adapts to the requested language.



What are good practices when prompting the AI to explain or debug code?

Provide the full code or relevant snippet; explain what the code is supposed to do; describe the issue or error message; ask for step-by-step explanation. Example: 'Here is my Python code. It should calculate factorial but returns wrong output. Explain what is wrong and how to fix it.' Clear context helps the model give precise debugging guidance.

How can you use prompts to generate structured outputs like JSON or HTML?

Explicitly enforce the output format: say 'Return only JSON' or 'Return valid HTML'; define keys or tags upfront; forbid extra explanations. Example JSON: 'Return only valid JSON. Fields: name, price, availability.' Example HTML: 'Generate a simple HTML table with columns: Name, Age, City. Do not include explanations.' This reduces formatting errors and extra text.



What is multimodal prompting?

Multimodal prompting uses more than one type of input: text, images, audio (in supported systems). The model combines these inputs to generate a response. Example: upload an image of a receipt and prompt 'Extract the total amount and date from this receipt.' The model uses both visual and text understanding.

How can prompt engineering help extract information from text?

Narrow the task, reduce ambiguity, enforce consistency.

Instead of 'Summarize this text', ask: 'From the text, extract: Key dates, Names of people, Decisions made. Return them as bullet points.' This targeted prompting improves accuracy and repeatability when extracting information.



How can you design prompts that reduce bias and generate safe outputs?

Set clear rules about fairness, neutrality, and scope: avoid leading or opinionated language; explicitly ask for balanced viewpoints; restrict assumptions about people or groups.

Example: 'Provide a neutral, factual explanation. Avoid stereotypes or assumptions. Base the answer on verified information only.'

How do you prompt an LLM to generate content in different writing styles?

Explicitly name the style and audience. Styles can include formal, casual, technical, marketing, academic, conversational.

Example: 'Explain cloud computing in a friendly, non-technical tone for a beginner.' Or: 'Rewrite the same explanation in a formal, academic style.' The content stays the same; tone and wording change.



How do you ensure prompts produce neutral and inclusive responses?

Instruct the model to avoid judgment, bias, and exclusion: ask for inclusive language; avoid absolute or emotional terms; request factual explanations. Example: 'Use inclusive and neutral language. Do not favor any group or viewpoint. Explain the topic objectively.' This helps the model focus on facts instead of opinions.

How can prompt constraints help avoid unsafe or incorrect outputs?

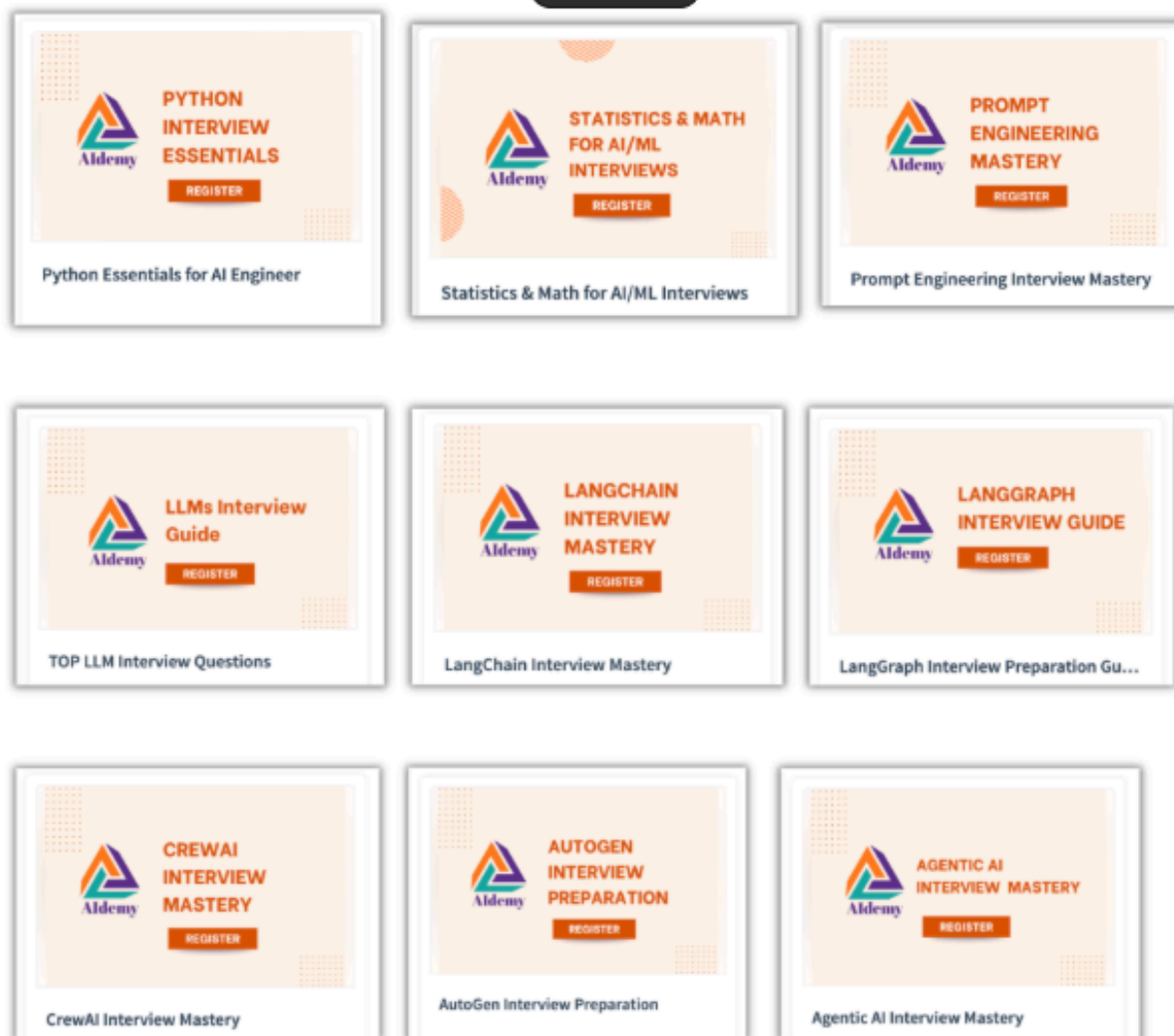
Constraints limit what the model is allowed to do: word or sentence limits, restricted topics, allowed sources, explicit refusal rules. Example: 'Answer only using the information provided. If the answer is not known, say I don't know. Do not speculate.' Constraints reduce hallucinations and unsafe guesses.



How do system-level instructions help control model behavior in sensitive tasks?

System-level instructions set global rules that override user input. They enforce safety, control tone, prevent misuse, ensure compliance. Example: 'You are a compliance assistant. Do not provide legal or medical advice. If asked, direct the user to a qualified professional.' This ensures consistent, safe behavior even when user prompts are risky.





AI Interview Mastery Bundle

13 modules. One place. Built for engineers who want to nail the interview.

Enroll now

[Link in caption](#)

FOLLOW ME



Chandra Sekhar

For more AI & ML content
